

Troféu

Nome do arquivo: `trofeu.c`, `trofeu.cpp`, `trofeu.pas`, `trofeu.java`, `trofeu.js` ou `trofeu.py`

Cinco alunos e alunas da escola conseguiram classificar-se para a Final da prestigiosa e muito difícil Competição Estadual de Programação, que será realizada no próximo mês.

Independentemente da classificação que os alunos da escola conseguirem na Final, a direção da escola decidiu que vai fazer uma premiação para os seus alunos. Quem conseguir a maior pontuação na Final, entre os alunos da escola, vai receber um troféu. E quem receber a segunda maior pontuação, entre os alunos da escola, vai receber uma placa comemorativa.

O problema é que pode haver alunos com as mesmas pontuações, de forma que dependendo dos resultados muitas combinações de prêmios são possíveis, como por exemplo, entre outros:

- cinco troféus (empate, todos com a mesma pontuação)
- um troféu (maior pontuação) e duas placas (empate na segunda maior pontuação)
- dois troféus (empate na maior pontuação) e duas placas (empate na segunda maior pontuação)

Dadas as pontuações dos cinco alunos e alunas, determine quantos troféus e placas deverão ser entregues.

Entrada

A entrada consiste de cinco linhas, cada uma contendo um inteiro P_i a pontuação de um aluno ou aluna. As pontuações serão dadas em ordem decrescente (ou seja, da maior para a menor pontuação).

Saída

Seu programa deve produzir uma única linha, contendo dois inteiros. O primeiro inteiro deve ser o número de troféus e o segundo inteiro o número de placas comemorativas a serem entregues.

Restrições

- $1 \leq P_i \leq 100$

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
100 90 80 70 60	1 1

Explicação do exemplo 1: A maior pontuação (100) ganha o troféu. A segunda maior pontuação (90) ganha a placa comemorativa.

Exemplo de entrada 2	Exemplo de saída 2
100 100 90 89 77	2 1

Explicação do exemplo 2: Há um empate na maior pontuação (100), portanto os dois ganham troféus. A segunda maior pontuação (90) ganha a placa comemorativa.

Exemplo de entrada 3	Exemplo de saída 3
99 99 99 99 99	5 0

Explicação do exemplo 3: Há um empate na maior pontuação (99), portanto os cinco ganham troféus. Não há entrega de placa comemorativa neste caso.

Caminho

Nome do arquivo: `caminho.c`, `caminho.cpp`, `caminho.pas`, `caminho.java`, `caminho.js` ou `caminho.py`

A pista de treinos de corridas da Prefeitura tem formato circular, com N postes igualmente espaçados na circunferência da pista, cada poste com exatamente uma lâmpada. Atualmente há lâmpadas de várias potências luminosas nos postes.

Vamos chamar o trecho de pista entre duas lâmpadas adjacentes (ou seja, uma vizinha à outra) de *trecho escuro* se a soma das potências luminosas dessas duas lâmpadas é menor do que 1000.

Para justificar um pedido à Prefeitura para que troquem as lâmpadas, os atletas querem saber qual o maior número de trechos escuros consecutivos (ou seja, um imediatamente em seguida de outro) da pista. Você pode ajudá-los?

Entrada

A primeira linha da entrada contém um inteiro N , o número de postes. Cada uma das N linhas seguintes contém um inteiro P_i , a potência luminosa de uma lâmpada. A posição de cada lâmpada é dada pela ordem da entrada (ou seja, a ordem das lâmpadas ao longo da pista é a ordem dada na entrada).

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o maior número de trechos escuros consecutivos.

Restrições

- $2 \leq N \leq 500\,000$
- $1 \leq P_i \leq 1\,000$ para $i \leq i \leq N$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 40 pontos, $P_1 + P_N \geq 1\,000$.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
3 499 500 499	3

Explicação do exemplo 1: O maior (e único) caminho de pares de lâmpadas com potência luminosa abaixo do especificado é formado pelos pares (499,500), (500, 499) e (499, 499), portanto a resposta é 3.

Exemplo de entrada 2	Exemplo de saída 2
6 900 700 100 900 200 700	1

Explicação do exemplo 2: O único par de lâmpadas com potência luminosa abaixo do especificado é (700,100), portanto a resposta é 1.

Exemplo de entrada 3	Exemplo de saída 3
6 500 800 290 700 200 400	4

Explicação do exemplo 3: O maior (e único caminho) com pares de lâmpadas com potência menor do que o especificado é formado pelos pares (290,700), (700,200), (200,400) e (400,500), portanto a resposta é 4.

Exemplo de entrada 4	Exemplo de saída 4
2 500 600	0

Explicação do exemplo 4: Não há nenhum par de lâmpadas com potência abaixo do especificado, portanto a resposta é 0.

Exemplo de entrada 5	Exemplo de saída 5
2 100 101	2

Explicação do exemplo 5: O maior (e único caminho) com pares de lâmpadas com potência menor do que o especificado é formado pelos pares (100,101) e (101,100), portanto a resposta é 2.

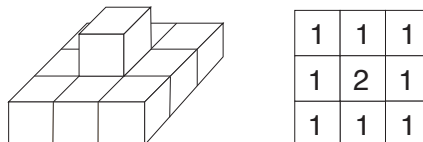
Pirâmide

Nome do arquivo: `piramide.c`, `piramide.cpp`, `piramide.pas`, `piramide.java`, `piramide.js` ou `piramide.py`

O rei da Nlogônia decidiu construir uma pirâmide no jardim do Palácio Real, usando cubos de pedra de mesmo tamanho. A *dimensão* de uma pirâmide é o número de cubos de pedra num dos lados da base (primeira camada) da pirâmide. A base da pirâmide é quadrada, ou seja, cada lado tem o mesmo número de cubos de pedra.

Na pirâmide, a partir da segunda camada, cada cubo de pedra deve ser empilhado exatamente em cima de outro cubo de pedra que não esteja na borda da camada abaixo. Além disso, o número de camadas deve ser o maior possível para uma dada dimensão, e em cada camada deve ser usado o maior número de cubos de pedra possível.

A figura abaixo à esquerda mostra uma pirâmide de dimensão 3; a figura à direita mostra o *plano de construção* para essa pirâmide, indicando quantos cubos de pedra devem ser empilhados em cada posição.



O rei ainda não decidiu qual a dimensão da pirâmide que vai construir, mas como é muito detalhista já avisou os Arquitetos Reais que antes de iniciar a construção eles devem produzir um plano de construção para a dimensão escolhida.

Ajude os Arquitetos Reais, escrevendo um programa que, dada a dimensão da pirâmide, produza o seu plano de construção.

Entrada

A primeira e única linha da entrada contém um número inteiro N , a dimensão da pirâmide.

Saída

Seu programa deve produzir o plano de construção da pirâmide, constituído por N linhas, cada linha contendo N números inteiros.

Restrições

- $1 \leq N \leq 100$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 10 pontos, $1 \leq N \leq 3$.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
3	1 1 1 1 2 1 1 1 1

Explicação do exemplo 1: Para uma pirâmide de dimensão 3, o maior número de camadas possível é 2.

Exemplo de entrada 2	Exemplo de saída 2
8	1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 1 1 2 3 3 3 3 2 1 1 2 3 4 4 3 2 1 1 2 3 4 4 3 2 1 1 2 3 3 3 3 2 1 1 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1

Explicação do exemplo 2: Para uma pirâmide de dimensão 8, o maior número de camadas possível é 4.

Tanque de combustível

Nome do arquivo: `tanque.c`, `tanque.cpp`, `tanque.pas`, `tanque.java`, `tanque.js` ou `tanque.py`

Cássio alugou um carro para a viagem de férias. O carro tem consumo de combustível constante (em quilômetros rodados por litro de combustível), independente da velocidade com que trafega. Ao fim da viagem, Cássio deve devolver o carro no aeroporto.

Cássio está terminando sua viagem de férias e está no momento na rodovia que leva ao aeroporto, em direção ao aeroporto para devolver o carro. Mais precisamente Cássio está no último posto de combustível existente na rodovia em que ele pode abastecer o carro antes de devolvê-lo.

Para economizar o máximo possível em combustível, Cássio quer devolver o carro com o menor número de litros possível no tanque – idealmente, com o tanque *zerado*, ou seja, sem combustível.

Dados o consumo do carro, a distância em que se encontra do aeroporto e a quantidade de combustível presente no tanque antes do abastecimento, determine qual deve ser a menor quantidade de combustível que Cássio deve comprar.

Entrada

A primeira linha contém um inteiro, C , o consumo do carro em quilômetros rodados por litro de combustível. A segunda linha contém um inteiro D , a distância do aeroporto, em quilômetros. A terceira linha contém um inteiro T , o número de litros de combustível presente no tanque antes do abastecimento. Você pode assumir que o tanque tem capacidade suficiente para armazenar todo o combustível que Cássio comprar.

Saída

Seu programa deve produzir uma única linha, contendo um único valor, com um dígito de precisão, indicando a quantidade de combustível que Cássio deve comprar, para chegar ao aeroporto com o tanque contendo a menor quantidade de combustível possível.

Restrições

- $1 \leq C \leq 50$
- $1 \leq D \leq 1000$
- $0 \leq T \leq 100$

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
2 10 0	5.0

Explicação do exemplo 1: O consumo é 2 km/l, Cássio está a 10 km do aeroporto e o tanque não tem combustível. Para chegar ao aeroporto o carro vai gastar 5.0 litros de combustível. Como o tanque não tem combustível, Cássio precisa comprar 5.0 litros de combustível.

Exemplo de entrada 2	Exemplo de saída 2
30 100 2	1.3

Explicação do exemplo 2: O consumo é 30 km/l, Cássio está a 100 km do aeroporto e o tanque tem 2 litros combustível. Para chegar ao aeroporto o carro vai gastar 3.33 litros de combustível. Como o tanque já tem 2 litros de combustível, Cássio precisa comprar 1.3 litros de combustível (note o arredondamento).

Exemplo de entrada 3	Exemplo de saída 3
50 120 3	0.0

Explicação do exemplo 3: O consumo é 50 km/l, Cássio está a 120 km do aeroporto e o tanque tem 3 litros combustível. Para chegar ao aeroporto o carro vai gastar 2.4 litros de combustível. Como o tanque já tem 3 litros de combustível, Cássio não precisa comprar combustível.

Exemplo de entrada 4	Exemplo de saída 4
50 73 0	1.5

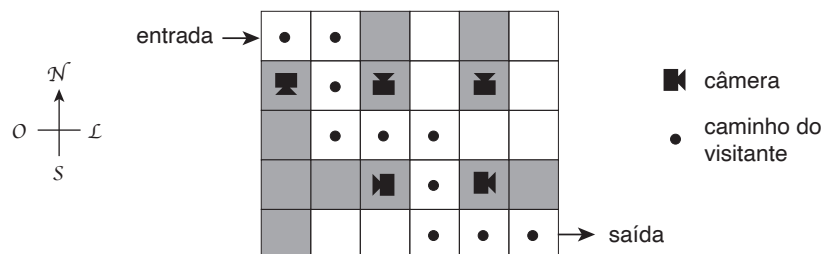
Explicação do exemplo 4: O consumo é 50 km/l, Cássio está a 73 km do aeroporto e o tanque não tem combustível. Para chegar ao aeroporto o carro vai gastar 1.46 litros de combustível. Como o tanque não tem combustível, Cássio precisa comprar 1.5 litros de combustível (note o arredondamento).

Câmeras

Nome do arquivo: `cameras.c`, `cameras.cpp`, `cameras.pas`, `cameras.java`, `cameras.js` ou `cameras.py`

Uma exposição vai ser montada num espaço retangular, dividido em $N \times M$ células dispostas em N colunas por M linhas. Uma *célula* é o espaço delimitado pela interseção de uma coluna com uma linha. As colunas estão na direção Norte-Sul e as linhas na direção Oeste-Leste. Para segurança das obras foram instaladas K câmeras, em células selecionadas. Cada câmera pode estar apontada para uma de quatro direções: Norte, Sul, Leste ou Oeste. Uma câmera observa todas as células da coluna ou linha na direção em que está apontada, a partir da célula em que está instalada (incluindo a célula em que está instalada).

A porta de entrada da exposição está na célula mais ao norte e mais à oeste, a porta de saída está na célula mais ao sul e mais ao leste. A figura abaixo ilustra um espaço de exposição com 6 colunas, 5 linhas e 5 câmeras instaladas.



Preocupado com a segurança, o organizador da exposição deseja saber se é possível que um visitante entre pela porta de entrada e saia pela porta de saída, movendo-se somente nas quatro direções (Norte, Sul, Leste ou Oeste) sem que seja observado por qualquer das câmeras instaladas.

Entrada

A primeira linha contém três inteiros N , M e K indicando respectivamente o número de colunas, o número de linhas e o número de câmeras instaladas. As colunas estão numeradas de 1 a N e as linhas estão numeradas de 1 a M . A coluna 1 é a coluna mais à Oeste e a linha 1 é a linha mais ao Norte. Cada uma das K linhas seguintes descreve uma câmera e contém dois inteiros C_i , L_i e um caractere D_i , indicando respectivamente a coluna, a linha e a direção em que a câmera está instalada. O caractere D_i pode ser N, S, L ou O, indicando respectivamente que a câmera está instalada direcionada para o Norte, Sul, Leste ou Oeste.

Saída

Seu programa deve produzir uma única linha, contendo um único caractere, que deve ser S se é possível que um visitante entre pela porta de entrada e saia pela porta de saída sem que seja observado por qualquer das câmeras instaladas, ou N caso contrário.

Restrições

- $2 \leq N \leq 30$; $2 \leq M \leq 30$; $1 \leq K \leq 30$
- $1 \leq C_i \leq N$, para $1 \leq i \leq K$
- $1 \leq L_i \leq M$, para $1 \leq i \leq K$
- D_i pode ser N, S, L ou O.

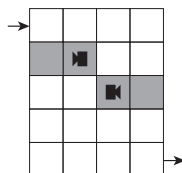
Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 10 pontos, $M = 2$ e $K = 1$.
- Para um conjunto de casos de testes valendo outros 10 pontos, $N = 3$, $M = 3$ e $K = 2$.
- Para um conjunto de casos de testes valendo outros 80 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
4 5 2 2 2 0 3 3 L	N

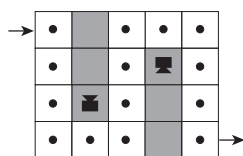
Explicação do exemplo 1:



Neste caso a resposta é Não.

Exemplo de entrada 2	Exemplo de saída 2
5 4 2 2 3 N 4 2 S	S

Explicação do exemplo 2:



Neste caso a resposta é Sim.

Exemplo de entrada 3	Exemplo de saída 3
6 5 5 1 2 S 3 2 N 5 2 N 3 4 0 5 4 L	S

Explicação do exemplo 3: Este caso é o exemplo dado no enunciado.

Subcadeias

Nome do arquivo: `subcadeias.c`, `subcadeias.cpp`, `subcadeias.pas`, `subcadeias.java`,
`subcadeias.js` ou `subcadeias.py`

Uma cadeia de caracteres é um *palíndromo* se os caracteres aparecem exatamente na mesma sequência quando lemos a cadeia da esquerda para a direita, ou da direita para a esquerda. Por exemplo, as cadeias `osso` e `arara` são palíndromos, mas as cadeias `xy` e `abbbab` não são palíndromos.

Uma *subcadeia* de uma dada cadeia de caracteres `um` é trecho contínuo da cadeia. Por exemplo, `abc`, `bc` e `d` são subcadeias de `abcde`, mas `abe` e `ded` não são.

O *comprimento* de uma cadeia de caracteres (ou subcadeia) é o número de caracteres da cadeia (ou subcadeia).

Dada uma cadeia de caracteres, determine o comprimento da maior subcadeia que é um palíndromo.

Entrada

A primeira linha da entrada contém um inteiro N , o comprimento da cadeia de caracteres. A segunda linha da entrada contém os N caracteres C_i que compõem a cadeia de caracteres.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o comprimento da maior subcadeia da cadeia da entrada que é um palíndromo.

Restrições

- $1 \leq N \leq 500$
- C_i é uma letra minúscula não acentuada, para $1 \leq i \leq N$.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
15 vovossorirmirim	5

Explicação do exemplo 1: As subcadeias que são palíndromos são: `v`, `o`, `s`, `r`, `i`, `m`, `ss`, `vov`, `ovo`, `rir`, `iri`, `osso`, `mirim`. A de maior comprimento é `mirim`, com comprimento 5.

Exemplo de entrada 2	Exemplo de saída 2
8 abxxxxba	8

Explicação do exemplo 2: As subcadeias que são palíndromos são: `a`, `b`, `x`, `xx`, `xxx`, `xxxx`, `bxxxxb`, `abxxxxba`. A de maior comprimento é `abxxxxba`, com comprimento 8.

Viagem

Nome do arquivo: `viagem.c`, `viagem.cpp`, `viagem.pas`, `viagem.java`, `viagem.js` ou `viagem.py`

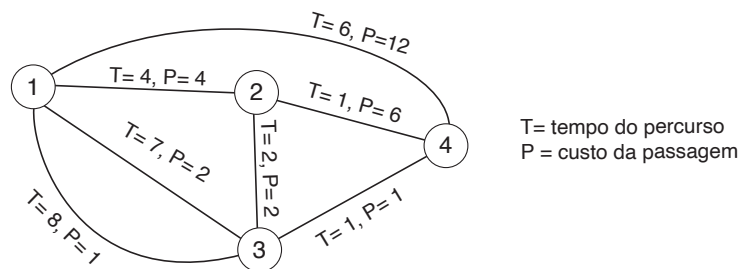
Você está viajando pelo arquipélago de Kiri, que é composto por um grande número de ilhas. Não há pontes entre as ilhas, de modo que a única maneira de viajar entre as ilhas é por navio.

Há várias rotas de navios disponíveis. Cada rota conecta duas ilhas distintas A e B e pode ser usada nas duas direções (de A para B ou de B para A). Cada rota tem um certo tempo de percurso (o mesmo nas duas direções) e um custo (o mesmo nas duas direções).

No momento você quer ir da ilha X para outra ilha Y , mas quer gastar no máximo um certo valor com a viagem. Você também está com pressa e gostaria de chegar o mais rapidamente possível ao seu destino.

Dados a lista das rotas disponíveis, com seus custos e tempos de percurso, escreva um programa para determinar se é possível chegar ao destino gastando no máximo o valor previsto para a viagem, e nesse caso qual o menor tempo para chegar ao destino. Note que pode não ser possível chegar ao destino, seja porque não há rota disponível ou porque o valor alocado para a viagem não é suficiente.

Por exemplo, considere o caso mostrado na figura abaixo, em que você está na ilha 1 e quer ir para a ilha 4:



1. Se o valor previsto é 10, a resposta é 5 e o caminho ótimo é $1 \rightarrow 2 \rightarrow 4$. Note que este caminho custa $4 + 6 = 10$ e demora tempo $4 + 1 = 5$.
2. Se o valor previsto é 7, a resposta é 7 e o caminho ótimo é $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, que custa $4 + 2 + 1 = 7$ e demora tempo $4 + 2 + 1 = 7$.
3. Se o valor previsto é 3, a resposta é 8 e o caminho ótimo é $1 \rightarrow 3 \rightarrow 4$, usando a aresta entre 1 e 3 que demora tempo 7 e tem custo 2. Note que este caminho custa $2 + 1 = 3$ e demora tempo $7 + 1 = 8$.
4. Se o valor previsto é 2, a resposta é 9 e o caminho ótimo é $1 \rightarrow 3 \rightarrow 4$, usando a aresta entre 1 e 3 que demora tempo 8 e tem custo 1, note que este caminho custa $1 + 1 = 2$ e demora tempo $8 + 1 = 9$.
5. Se o valor previsto é 1, não existe caminho que satisfaça as restrições, por isso a resposta é -1 .

Entrada

A primeira linha da entrada contém três inteiros V , N e M , respectivamente o valor disponível para a viagem, o número de ilhas e o número de rotas. As ilhas são identificadas por inteiros de 1 a N .

Cada uma das M linhas seguintes descreve uma rota e contém quatro inteiros A_i , B_i , T_i e P_i , onde A_i e B_i representam ilhas, T_i o tempo de percurso e P_i o custo de uma passagem para essa rota. A última linha da entrada contém dois inteiros X e Y , o início e o destino da sua viagem.

Saída

Seu programa deve produzir uma única linha na saída, que deve conter um único inteiro, o menor tempo necessário para chegar ao destino, ou o valor -1 caso não seja possível chegar ao destino.

Restrições

- $2 \leq N \leq 10\,000$
- $1 \leq M \leq 2\,000$
- $1 \leq V \leq 200$
- $1 \leq A_i, B_i \leq N$, $A_i \neq B_i$, para $1 \leq i \leq M$.
- Pode haver mais de uma rota entre o mesmo par de ilhas.
- $1 \leq T_i \leq 100\,000$, para $1 \leq i \leq M$.
- $0 \leq P_i \leq 200$, para $1 \leq i \leq M$.
- $1 \leq X, Y \leq N$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $N \leq 200$ e $P_i = 0$ para $1 \leq i \leq M$.
- Para um conjunto de casos de testes valendo outros 10 pontos, $N \leq 10\,000$ e $P_i = 0$ para $1 \leq i \leq M$.
- Para um conjunto de casos de testes valendo outros 30 pontos, $N \leq 100$ e $V \leq 10$.
- Para um conjunto de casos de testes valendo outros 40 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 10 4 7 1 2 4 4 1 3 7 2 3 1 8 1 3 2 2 2 4 2 1 6 3 4 1 1 1 4 6 12 1 4	Exemplo de saída 1 5
Exemplo de entrada 2 3 3 3 1 2 5 2 3 2 8 2 1 3 1 4 1 3	Exemplo de saída 2 -1